Chapter 7

GRPO — Group Relative Policy Optimization

Group Relative Policy Optimization (GRPO) [323] is a reinforcement learning algorithm designed specifically for language models that eliminates the need for a separate value network (critic). Introduced by DeepSeek as part of their DeepSeekMath work and later scaled to DeepSeek-R1 [72], GRPO has rapidly become the dominant RL method for LLM training—adopted by most open-source alignment frameworks (TRL, OpenRLHF, veRL) as the default algorithm.

The core idea is deceptively simple: instead of training a neural network to predict expected reward (the critic in PPO), GRPO *estimates* it empirically by generating multiple responses to the same prompt and using the group's reward statistics as a baseline. This removes an entire model from memory, halves the engineering complexity, and—surprisingly—often outperforms PPO because empirical baselines are more accurate than a poorly-trained value function.

GRPO is particularly effective for:

- **Reasoning tasks** with verifiable rewards (math, code) where binary correctness provides a clean signal.
- **Large models** (70B+) where the memory savings from removing the critic are critical.
- **Multi-turn and agentic settings** where value estimation across tool calls is intractable.

This chapter covers GRPO's motivation, algorithm, key variants (Dr. GRPO, DAPO, 2-GRPO, GDPO), and practical implementation with TRL.

7.1 Motivation

PPO's value model (critic) has three major problems for language:

1. **Memory**: The value head shares the policy backbone (140GB for 70B). Doubles memory if separate.
2. **Accuracy**: Predicting expected reward for a partial sequence is extremely hard. The value function is often wrong → wrong advantages → wrong gradient direction.
3. **Training**: Value head needs many samples to converge. During early RL, it gives noisy predictions that destabilize policy learning.

GRPO's key insight [323]: Instead of learning $V(s)$, *estimate* it empirically from a group of samples. Generate G responses to the same prompt, compute their rewards, and use the group statistics as the baseline.

7.2 Algorithm

1. For each prompt x, sample G completions: $\{y_1, \ldots, y_G\} \sim \pi_\theta(\cdot|x)$
2. Score each: $r_i = R(x, y_i)$
3. Normalize within group: $\hat{A}_i = \frac{r_i - \mu_G}{\sigma_G}$ where $\mu_G = \frac{1}{G}\sum_j r_j$, $\sigma_G = \text{std}(\{r_j\})$
4. Apply PPO-style clipped update using these advantages

$$\hat{A}_i = \frac{r_i - \mu_G}{\sigma_G}, \qquad L = \mathbb{E}\left[\min\left(r_t(\theta)\hat{A}_i,\ \text{clip}(r_t(\theta), 1\pm\epsilon)\hat{A}_i\right)\right] - \beta D_{\text{KL}}[\pi_\theta \| \pi_{\text{ref}}] \tag{7.1}$$

Why Group Normalization Works

The group mean approximates $V(s)$: If you sample enough responses to the same prompt, their average reward is a Monte Carlo estimate of the expected reward = value function.
Above mean = good move: $\hat{A}_i > 0$ means this response is better than average for this prompt. Reinforce it.
Below mean = bad move: $\hat{A}_i < 0$ means worse than average. Suppress it.
Normalization: Dividing by σ_G ensures advantages are scale-invariant across prompts with different reward ranges.
DeepSeek-R1 breakthrough [72]: Pure GRPO with binary correctness rewards ($r = 1$ if answer correct, $r = 0$ otherwise) trained on math/code spontaneously developed chain-of-thought reasoning, self-verification, and error correction — without any explicit instruction to do so.

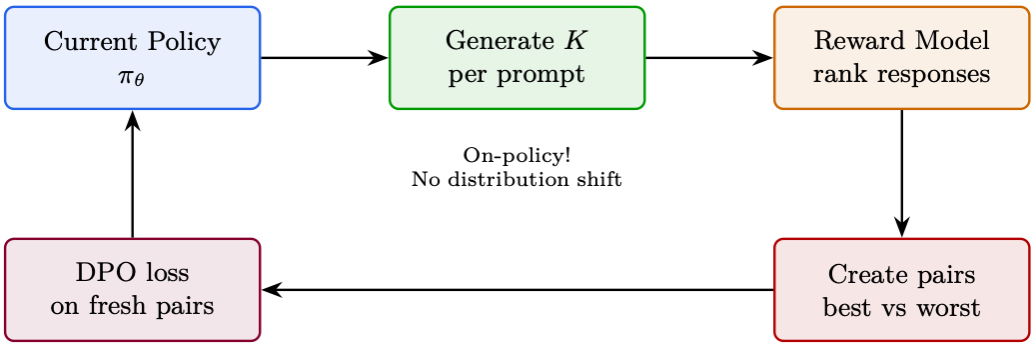

Figure 7.1: GRPO in action: G=5 responses are sampled for a single math prompt. Three are correct (r=1), two are wrong (r=0). The group mean μ_G=0.6 acts as the baseline; correct responses receive positive advantage (reinforced), wrong ones receive negative advantage (suppressed).

7.3 TRL Implementation

The following shows a minimal working example using HuggingFace TRL.

```
from trl import GRPOConfig, GRPOTrainer
from transformers import AutoModelForCausalLM, AutoTokenizer

model = AutoModelForCausalLM.from_pretrained("Qwen/Qwen2.5-7B-Instruct",
    torch_dtype=torch.bfloat16, attn_implementation="flash_attention_2")
tokenizer = AutoTokenizer.from_pretrained("Qwen/Qwen2.5-7B-Instruct")

grpo_config = GRPOConfig(
    output_dir="./grpo_output",
    num_generations=8,              # G = group size
    temperature=1.0,                # High temp for diversity within group
    max_completion_length=2048,     # Max response length
    beta=0.04,                      # KL penalty coefficient
    learning_rate=1e-6,
```

```
    per_device_train_batch_size=2,  # Prompts per device (x8 gens = 16 responses)
    gradient_accumulation_steps=8,
    num_train_epochs=2,
    bf16=True,
    gradient_checkpointing=True,
    max_grad_norm=0.5,
    logging_steps=10,
    # vLLM generation for speed (critical for GRPO due to 8x generation)
    use_vllm=True,
    vllm_gpu_memory_utilization=0.7,
)

# Reward function: binary correctness for math
def reward_fn(completions, prompts, **kwargs):
    """Return list of floats: 1.0 if correct, 0.0 if wrong."""
    rewards = []
    for completion, prompt in zip(completions, prompts):
        answer = extract_answer(completion)
        expected = get_ground_truth(prompt)
        rewards.append(1.0 if answer == expected else 0.0)
    return rewards

# Can combine multiple reward functions!
def format_reward_fn(completions, **kwargs):
    """Bonus for using proper LaTeX formatting."""
    return [0.5 if "\\boxed{" in c else 0.0 for c in completions]

trainer = GRPOTrainer(
    model=model,
    args=grpo_config,
    reward_funcs=[reward_fn, format_reward_fn],  # Multi-objective!
    train_dataset=math_dataset,
    tokenizer=tokenizer,
)
trainer.train()
```

7.4 Group Size Analysis

G	Signal Quality	Compute	When to Use
2	Very noisy (coin flip)	Low	Never recommended — too noisy for stable learning
4	Moderate	Moderate	Quick experiments, easy tasks (pass rate $> 50\%$)
8	Good (standard)	High	Default. Good balance for most tasks
16	Excellent	Very high	Hard tasks (pass rate $< 20\%$), need many attempts to get positives
32	Near-perfect	Extreme	Only if you have massive compute and very hard task

Critical: Group Must Contain Both Successes and Failures

If all G responses are correct ($r_i = 1\ \forall i$): all advantages $= 0$, no learning signal!

If all wrong: same problem. The prompt's difficulty must match model's capability.

Goldilocks rule: Filter prompts to 20–80% pass rate for current model. Re-filter every 500 steps as model improves.

7.5 GRPO Variants and Extensions

7.5.1 Diversity in GRPO Groups

Mode Collapse in RL Training

Without diversity pressure, RL-trained LLMs collapse to a narrow set of high-reward responses:

- The model learns one "template" answer for each question type
- Entropy drops rapidly; the model becomes deterministic
- Reward hacking becomes easier (narrow outputs are easier to exploit)
- Generalization suffers: the model memorizes reward patterns, not reasoning

The KL penalty $\beta D_{\text{KL}}[\pi_\theta \| \pi_{\text{ref}}]$ is the primary diversity mechanism, but it's not sufficient alone.

GRPO Group Diversity

GRPO generates N responses per prompt and uses within-group ranking. Diversity within the group is critical:

- **High temperature** ($\tau = 0.8$–1.0): Ensures varied responses for meaningful comparison
- **Large** N (8–16): More samples = more likely to include both good and bad approaches
- **DAPO's "No Repeat" penalty**: Rejects duplicate responses within a group to force exploration
- If all N responses are identical: advantage is zero, no learning signal
- If responses are too diverse (random): reward signal is noisy, slow learning

Sweet spot: Temperature that gives distinct approaches while staying on-topic.

Table 7.1: Diversity-promoting methods for RL training.

Method	How It Promotes Diversity
Entropy bonus	Add $\alpha H(\pi_\theta)$ to the reward. Directly penalizes low-entropy (deterministic) policies.
KL penalty	$-\beta D_{\text{KL}}[\pi_\theta \| \pi_{\text{ref}}]$ prevents collapse toward a single mode.
Rejection sampling	Generate many candidates, keep top-k by reward. Naturally selects for diverse high-quality responses.
Best-of-N	At inference: generate N responses, score all, return the best. Diversity comes from sampling.
DPO with diverse pairs	Train on pairs where chosen/rejected differ in *approach*, not just quality.
Multi-reward	Use multiple reward models (safety, helpfulness, code quality). Prevents collapsing to one dimension.

Diversity vs. Quality Tradeoff

More diversity is not always better:

- Too much diversity (high entropy) = random, unhelpful responses
- Too little diversity (low entropy) = repetitive, reward-hacked responses
- **Monitor**: Track response entropy, unique n-gram ratio, and reward distribution width during training. If all three are dropping simultaneously, you have a collapse problem.

Verbalized Sampling for RL Data Collection

Post-training alignment (RLHF, DPO) often reduces output diversity due to *typicality bias*: human annotators systematically prefer familiar, "typical" text over novel alternatives. This mode collapse is a data-level phenomenon, not purely algorithmic.

Verbalized Sampling (VS) [422] is a training-free prompting strategy that circumvents this collapse by asking the model to **explicitly verbalize a probability distribution** over multiple responses in a single generation.

Verbalized Sampling – Core Idea

Instead of sampling a single response (which collapses to the mode), prompt the model to output *multiple candidate responses along with their probabilities*:
`"Generate 5 jokes about coffee and their corresponding probabilities."`
The model produces a list like:

1. Joke A (probability: 0.35)
2. Joke B (probability: 0.25)
3. Joke C (probability: 0.20)
4. Joke D (probability: 0.12)
5. Joke E (probability: 0.08)

Then sample from this verbalized distribution. Because the model explicitly represents the full distribution (not just the argmax), lower-probability but creative/diverse responses become accessible.

```python
# Verbalized Sampling: prompt model to output distribution
def verbalized_sample(model, tokenizer, task, n=5):
    prompt = (
        f"{task}\n\n"
        f"Generate {n} different responses and assign a probability "
        f"to each (probabilities should sum to 1.0). "
        f"Format: [response] (probability: X.XX)"
    )
    output = model.generate(
        tokenizer(prompt, return_tensors="pt").input_ids,
        max_new_tokens=1024,
        temperature=0.7,
        do_sample=True,
    )
    # Parse responses and probabilities from output
    responses, probs = parse_verbalized_distribution(
        tokenizer.decode(output[0])
    )
    # Sample from the verbalized distribution
    import random
    chosen = random.choices(responses, weights=probs, k=1)[0]
    return chosen
```

Why Verbalized Sampling Works

- **Bypasses mode collapse**: Standard sampling from aligned models heavily concentrates on one or two "safe" responses. VS forces the model to articulate alternatives it *knows* but wouldn't normally surface.
- **Diversity is semantic**: Unlike temperature scaling (lexical noise), VS produces genuinely different approaches—the model reasons about distinct options.
- **Scales with capability**: More capable models produce better-calibrated verbalized

distributions—they benefit *more* from VS (1.6–2.1× diversity gain in creative writing).

- **Training-free**: No fine-tuning or modified decoding; works with any instruction-following model at inference time.
- **For GRPO**: Use VS to generate the G response candidates per prompt—ensures the group contains semantically diverse approaches rather than surface-level variations.

Before diving into the extensions, let us briefly recap the base GRPO algorithm established in the previous sections. The core mechanism—sampling a group of completions, normalizing their rewards, and applying a clipped policy gradient—is elegant in its simplicity. However, practitioners quickly discovered specific failure modes: pretraining bias diluting gradients (Dr. GRPO), symmetric clipping limiting exploration (DAPO), wasteful large group sizes (2-GRPO), and reward-scale imbalance in multi-objective settings (GDPO). The following sections address each of these in turn.

GRPO Baseline Recap

Given a prompt q, sample G completions $\{o_1, \ldots, o_G\}$ from the current policy π_θ. Compute rewards $\{r_1, \ldots, r_G\}$ and normalise:

$$\hat{A}_i = \frac{r_i - \mu_r}{\sigma_r + \epsilon}, \qquad \mu_r = \frac{1}{G}\sum_{i=1}^{G} r_i, \quad \sigma_r = \sqrt{\frac{1}{G}\sum_{i=1}^{G}(r_i - \mu_r)^2}.$$

The clipped surrogate loss (per token) is:

$$\mathcal{L}_{\text{GRPO}} = -\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\min\Big(\rho_{i,t}\,\hat{A}_i,\ \text{clip}(\rho_{i,t}, 1{-}\epsilon, 1{+}\epsilon)\,\hat{A}_i\Big),$$

where $\rho_{i,t} = \pi_\theta(o_{i,t}|q, o_{i,<t}) \,/\, \pi_{\text{old}}(o_{i,t}|q, o_{i,<t})$.

7.5.2 DAPO – Dynamic Adaptive Policy Optimization

Why DAPO?

Base GRPO uses *symmetric* clipping: the policy is equally constrained whether it wants to increase or decrease the probability of a token. But exploration and exploitation have different risk profiles. Increasing the probability of a good token is generally safe; suppressing a token that happened to appear in a bad completion can be catastrophically wrong if the token itself is neutral. DAPO [413] introduces five targeted fixes that together substantially improve training stability and final performance.

Component 1 – Asymmetric Clipping (Clip-Higher)

Standard PPO/GRPO clips the importance ratio symmetrically at $[1-\epsilon, 1+\epsilon]$. DAPO replaces this with an asymmetric band:

$$\boxed{\text{clip}_{\text{DAPO}}(\rho, A) = \begin{cases} \text{clip}(\rho,\ 1-\epsilon,\ 1+\epsilon_{\text{high}}) & \text{if } A > 0 \\ \text{clip}(\rho,\ 1-\epsilon,\ 1+\epsilon) & \text{if } A \le 0 \end{cases}}$$

where $\epsilon_{\text{high}} > \epsilon$ (typical values: $\epsilon = 0.2$, $\epsilon_{\text{high}} = 0.28$). When the advantage is positive the policy is allowed to move further toward the good token; when the advantage is negative the usual conservative clipping applies to avoid over-suppression.

Component 2 – Token-Level Loss Aggregation

Base GRPO divides the loss by the *number of sequences* G. DAPO divides by the *total number of tokens* across all sequences:

$$\mathcal{L}_{\text{token}} = -\frac{1}{\sum_{i=1}^{G}|o_i|}\sum_{i=1}^{G}\sum_{t=1}^{|o_i|}\min(\rho_{i,t}\hat{A}_i,\ \text{clip}_{\text{DAPO}}(\rho_{i,t},\hat{A}_i)\,\hat{A}_i).$$

This prevents long completions from dominating the gradient signal simply because they contain more tokens.

Component 3 – Overlong Filtering

When a completion is truncated (no EOS token within the maximum length budget), it provides *misleading* signal: the model is penalised for tokens that were generated correctly but happened to appear before the truncation boundary. DAPO masks these completions entirely:

$$m_i = \mathbf{1}[\text{EOS} \in o_i], \qquad \mathcal{L}_{\text{filtered}} = -\frac{\sum_{i=1}^{G} m_i \sum_t(\cdots)}{\sum_{i=1}^{G} m_i|o_i|}.$$

Component 4 – Soft Overlong Punishment

Rather than a hard mask, a softer variant applies a length penalty that grows smoothly as completions approach the maximum length $L_{\max}$:

$$r_i \leftarrow r_i - \lambda \cdot \max\left(0,\ \frac{|o_i| - L_{\text{cache}}}{L_{\max} - L_{\text{cache}}}\right),$$

where L_{cache} is a "safe" length threshold.

Component 5 – Dynamic Sampling

DAPO re-samples prompts whose entire group of completions receives the same reward (all correct or all incorrect), because such groups contribute zero gradient after normalisation. This keeps the effective batch size stable throughout training.

DAPO in TRL

```python
from trl import GRPOConfig, GRPOTrainer

config = GRPOConfig(
    # Asymmetric clipping
    epsilon=0.2,
    epsilon_high=0.28,          # Clip-Higher
    # Token-level loss
    loss_type="dapo",           # enables token-level aggregation
    # Overlong filtering
    mask_truncated_completions=True,
    # Generation budget
    max_completion_length=1024,
    num_generations=8,
    # Note: DAPO loss internally handles zero-variance group filtering
)

trainer = GRPOTrainer(
    model=model,
    reward_funcs=[reward_fn],
    args=config,
    train_dataset=dataset,
)
trainer.train()
```

When to Use DAPO

- Long-form reasoning tasks where completions frequently hit the length limit.

- Any setting where you observe reward variance collapsing mid-training.
- When base GRPO shows instability (loss spikes, entropy collapse).
- Recommended as a *drop-in improvement* over base GRPO for most tasks.

7.5.3 GSPO – Group Sequence Policy Optimization

The Off-Policy Problem

GRPO clips importance ratios *per token*. But a sequence of 500 tokens can have a product of per-token ratios that is astronomically large or small, even if each individual ratio is within $[1-\epsilon, 1+\epsilon]$. When training for multiple gradient steps on the same batch (off-policy), this mismatch grows rapidly and the clipping bound becomes meaningless at the sequence level.

GSPO [51] defines a *sequence-level* importance weight as the geometric mean of per-token ratios, which equals the $|o_i|$-th root of the full sequence probability ratio:

$$s_i(\theta) = \left(\frac{\pi_\theta(o_i \mid q)}{\pi_{\text{old}}(o_i \mid q)}\right)^{1/|o_i|} = \exp\left(\frac{1}{|o_i|}\sum_{t=1}^{|o_i|}\log\frac{\pi_\theta(o_{i,t}|q,o_{i,<t})}{\pi_{\text{old}}(o_{i,t}|q,o_{i,<t})}\right).$$

This is the *length-normalised* sequence probability ratio. The GSPO loss clips this single scalar per sequence:

$$\mathcal{L}_{\text{GSPO}} = -\frac{1}{G}\sum_{i=1}^{G}\min\Big(s_i(\theta)\,\hat{A}_i,\ \text{clip}(s_i(\theta), 1-\epsilon, 1+\epsilon)\,\hat{A}_i\Big).$$

GSPO vs GRPO Clipping

- **GRPO**: clips each of the $|o_i|$ per-token ratios independently. A sequence can have all ratios within bounds yet have a product ratio of 10^{50}.
- **GSPO**: clips the geometric mean once per sequence. Guarantees the *sequence-level* policy shift is bounded.
- GSPO is theoretically correct for off-policy IS; GRPO is an approximation.

GSPO in TRL

```
from trl import GRPOConfig, GRPOTrainer

config = GRPOConfig(
    # Sequence-level importance sampling
    importance_sampling_level="sequence",   # GSPO mode
    # Off-policy: reuse each batch for multiple gradient steps
    steps_per_generation=4,
    num_generations=8,
    epsilon=0.2,
)

trainer = GRPOTrainer(
    model=model,
    reward_funcs=[reward_fn],
    args=config,
    train_dataset=dataset,
)
```

When to Use GSPO

GSPO is most beneficial when `steps_per_generation > 1` (off-policy training). For purely on-policy training (steps_per_generation = 1) the difference from GRPO is negligible. Off-policy

training can dramatically reduce generation cost (the most expensive step), making GSPO + off-policy a strong efficiency choice.

7.5.4 Dr. GRPO – Debiased Reward GRPO

The Pretraining Bias Problem

Standard GRPO normalises advantages within a group, but the *pretraining distribution* introduces a systematic bias: tokens that are common in pretraining data receive large gradients even when they carry no task-relevant information. Dr. GRPO [232] identifies and corrects this bias, focusing gradient signal on *informative* tokens.

Dr. GRPO modifies the per-token gradient weight to account for the token's marginal contribution to the reward signal. Tokens that the model already assigns high probability to (regardless of the reward) are down-weighted:

$$w_{i,t} = \hat{A}_i \cdot (1 - \pi_{\text{ref}}(o_{i,t}|q, o_{i,<t})),$$

where π_{ref} is the reference (pretrained) model. This is a form of *token efficiency*: the gradient is concentrated on tokens where the policy genuinely needs to change.

Dr. GRPO in TRL

```
from trl import GRPOConfig, GRPOTrainer

config = GRPOConfig(
    loss_type="dr_grpo",
    num_generations=8,
    beta=0.04,   # KL penalty coefficient
)

trainer = GRPOTrainer(
    model=model,
    ref_model=ref_model,   # required for token weighting
    reward_funcs=[reward_fn],
    args=config,
    train_dataset=dataset,
)
```

When to Use Dr. GRPO

- When training on tasks with a large vocabulary mismatch between pretraining and RL.
- When you observe that common filler tokens dominate the gradient.
- Pairs well with a reference model that is close to the initial policy.

7.5.5 2-GRPO – Minimal Two-Rollout GRPO

"It Takes Two" Insight

The "It Takes Two" paper [399] demonstrates empirically and theoretically that GRPO with $G = 2$ (just two completions per prompt) matches or exceeds GRPO with $G = 16$ on most reasoning benchmarks. This is surprising – why would fewer samples be sufficient?

The key insight is that GRPO's effectiveness does *not* primarily come from accurate advantage estimation (which requires large G). Instead, it comes from an implicit *contrastive objective* that is structurally similar to DPO:

$$\mathcal{L}_{\text{2-GRPO}} \approx -\mathbb{E}_{(o^+,o^-)\sim\pi_\theta}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(o^+|q)}{\pi_{\text{old}}(o^+|q)} - \beta\log\frac{\pi_\theta(o^-|q)}{\pi_{\text{old}}(o^-|q)}\right)\right],$$

where o^+ is the higher-reward completion and o^- the lower-reward one. With $G = 2$, this contrastive structure is explicit. With $G = 16$, the same signal is present but diluted by redundant pairs.

Compute Savings from 2-GRPO

- $G = 2$ vs $G = 16$: **8× less generation compute**.
- Generation is typically the bottleneck (60–80% of wall-clock time).
- Total training speedup: approximately 4–6× end-to-end.
- No accuracy loss on GSM8K, MATH, and code benchmarks.

2-GRPO in TRL

```
from trl import GRPOConfig, GRPOTrainer

config = GRPOConfig(
    num_generations=2,       # The key change -- just two rollouts
    loss_type="grpo",        # Standard GRPO loss is fine
    epsilon=0.2,
    # With G=2, batch size must be at least 2 * num_prompts_per_step
    per_device_train_batch_size=2,
)

trainer = GRPOTrainer(
    model=model,
    reward_funcs=[reward_fn],
    args=config,
    train_dataset=dataset,
)
```

Caveats of 2-GRPO

With $G = 2$, advantage normalisation is over only two values, so the normalised advantages are always $\{+1, -1\}$ (or $\{0, 0\}$ if rewards are equal). This means the gradient magnitude is fixed regardless of the reward gap. For tasks where the *magnitude* of the reward difference matters (e.g., partial credit), larger G may still be beneficial.

7.5.6 SAPO – Soft Adaptive Policy Optimization

The Brittleness of Hard Clipping

PPO-style clipping creates a discontinuous gradient: the gradient is zero outside the clip band and non-zero inside. This "cliff edge" can cause instability near the boundary and makes the trust region sensitive to the choice of ϵ. SAPO [131] replaces the hard clip with a smooth, temperature-controlled gate function.

SAPO replaces the $\min(\rho A, \text{clip}(\rho, \cdot)\, A)$ objective with a smooth surrogate:

$$\mathcal{L}_{\text{SAPO}}(\rho, A) = \begin{cases} -A \cdot \sigma\left(\dfrac{\rho - 1}{\tau_+}\right) \cdot \rho & \text{if } A > 0 \\ -A \cdot \sigma\left(\dfrac{1 - \rho}{\tau_-}\right) \cdot \rho & \text{if } A \leq 0 \end{cases}$$

where σ is the sigmoid function and τ_+, τ_- are asymmetric temperature parameters. A higher temperature produces a softer gate (more exploration); a lower temperature approaches hard clipping.

SAPO Temperature Intuition

- $\tau_+ = 1.0$: moderate gate for positive advantages (allow exploration).
- $\tau_- = 1.05$: slightly softer gate for negative advantages (avoid over-suppression).

- As $\tau \to 0$: recovers hard PPO clipping.
- As $\tau \to \infty$: recovers unclipped policy gradient.

SAPO in TRL

```
from trl import GRPOConfig, GRPOTrainer

config = GRPOConfig(
    loss_type="sapo",
    sapo_temperature_pos=1.0,     # tau_+ for positive advantages
    sapo_temperature_neg=1.05,    # tau_- for negative advantages
    num_generations=8,
)

trainer = GRPOTrainer(
    model=model,
    reward_funcs=[reward_fn],
    args=config,
    train_dataset=dataset,
)
```

7.5.7 TIS and MIS – Truncated and Masked Importance Sampling

The Silent vLLM Probability Mismatch

When using vLLM for fast generation, the log-probabilities returned by vLLM differ from those computed during the training forward pass [436]. This is *not* a bug – it arises from different CUDA kernels, different floating-point precision, and different attention implementations (e.g., FlashAttention vs PagedAttention). The mismatch silently breaks the on-policy assumption: the "old policy" probabilities used to compute importance ratios are wrong, leading to biased gradient estimates.

Truncated Importance Sampling (TIS)

TIS corrects the bias by multiplying the gradient by a truncated correction factor:

$$\boxed{w_{\text{TIS}}(o_i) = \min\left(C,\ \frac{\pi_{\text{train}}(o_i|q)}{\pi_{\text{vllm}}(o_i|q)}\right),}$$

where π_{train} is the probability from the training forward pass and π_{vllm} is the probability reported by vLLM. The truncation at C prevents extreme corrections from destabilising training.

Masked Importance Sampling (MIS)

MIS takes a harder approach: it zeros out the gradient for any sequence where the correction ratio exceeds a threshold C:

$$w_{\text{MIS}}(o_i) = \mathbf{1}\left[\frac{\pi_{\text{train}}(o_i|q)}{\pi_{\text{vllm}}(o_i|q)} \le C\right].$$

This is more conservative but avoids the risk of large (even truncated) correction weights.

Sequence-Level vs Token-Level IS

Both TIS and MIS can be applied at the token level or the sequence level:

- **Sequence-level**: compute the ratio as the geometric mean over all tokens (as in GSPO). Theoretically correct but higher variance.

- **Token-level**: compute a separate ratio for each token. Biased (the product of per-token corrections is not the sequence correction) but lower variance.

TIS and MIS in TRL

```
from trl import GRPOConfig, GRPOTrainer

# Truncated IS correction for vLLM probability mismatch
config_tis = GRPOConfig(
    use_vllm=True,
    vllm_importance_sampling_correction=True,
    vllm_importance_sampling_mode="sequence_truncate",  # TIS
    vllm_importance_sampling_cap=5.0,                   # C threshold
)

# Masked IS correction
config_mis = GRPOConfig(
    use_vllm=True,
    vllm_importance_sampling_correction=True,
    vllm_importance_sampling_mode="sequence_mask",      # MIS
    vllm_importance_sampling_cap=3.0,
)
```

When to Use TIS/MIS

- **Always** consider enabling when using vLLM for generation.
- TIS is preferred when the mismatch is small (same model, different precision).
- MIS is preferred when the mismatch is large or unpredictable.
- Sequence-level IS is theoretically preferred; token-level is a practical compromise.

7.5.8 VESPO – Variational Sequence-Level Soft Policy Optimization

Principled Reward Reshaping

Most GRPO variants modify the clipping mechanism heuristically. VESPO derives a principled reward-reshaping kernel from a variational inference framework, treating policy optimisation as approximate posterior inference. VESPO [243] derives a resulting kernel that is smooth, asymmetric, and naturally handles staleness in asynchronous or off-policy training.

VESPO derives a weighting function $W(\tau)$ for each trajectory τ from the variational objective. The final gradient weight takes the form:

$$\boxed{g(\tau) = W(\tau)^k \cdot \exp\big(\lambda(1 - W(\tau))\big),}$$

where $W(\tau) = \pi_\theta(\tau)/\pi_{\text{old}}(\tau)$ is the sequence-level importance weight, k controls the sharpness of the weighting, and λ controls the exponential decay for stale (low-weight) trajectories. This kernel:

- Is smooth everywhere (no discontinuous gradient at clip boundaries).
- Naturally down-weights stale trajectories ($W \ll 1$) via the exponential term.
- Is asymmetric: high-weight trajectories ($W > 1$) are treated differently from low-weight ones.

VESPO in TRL

```
from trl import GRPOConfig, GRPOTrainer

config = GRPOConfig(
    loss_type="vespo",
```

```
    vespo_k_pos=2.0,          # sharpness exponent (positive advantages)
    vespo_lambda_pos=3.0,     # staleness decay (positive advantages)
    num_generations=8,
    steps_per_generation=2,   # off-policy; VESPO handles staleness
)

trainer = GRPOTrainer(
    model=model,
    reward_funcs=[reward_fn],
    args=config,
    train_dataset=dataset,
)
```

7.5.9 DPPO – Direct Policy Divergence Policy Optimization

The Problem with Ratio Clipping

PPO's ratio clipping is a proxy for constraining the KL divergence between old and new policy. But the proxy is imperfect: clipping over-penalises low-probability tokens (where a small absolute change in probability corresponds to a large ratio change) and under-penalises high-probability tokens (where a large absolute change corresponds to a small ratio change). DPPO [7] replaces ratio clipping with *direct divergence estimates*.

DPPO computes the trust region constraint directly using either Total Variation (TV) or KL divergence between the old and new policy distributions:

$$\mathcal{L}_{\text{DPPO}} = -\mathbb{E}\Big[\hat{A} \cdot \pi_\theta(o|q) \cdot \mathbf{1}[D(\pi_\theta \| \pi_{\text{old}}) \leq \delta]\Big],$$

where D is the chosen divergence measure. In practice, DPPO approximates this with token-level binary or top-k masks:

- **binary_tv**: mask tokens where $|\pi_\theta - \pi_{\text{old}}| > \delta$.
- **binary_kl**: mask tokens where $\pi_\theta \log(\pi_\theta / \pi_{\text{old}}) > \delta$.
- **topk_tv**: keep only the top-k tokens by TV contribution.
- **topk_kl**: keep only the top-k tokens by KL contribution.

DPPO – Conceptual Implementation

DPPO is not yet available as a built-in TRL trainer. A custom implementation would use GRPOTrainer with a modified loss that clips based on distributional divergence (TV or KL) rather than the standard probability ratio:

```
# Pseudocode: DPPO requires a custom trainer subclass
from trl import GRPOConfig, GRPOTrainer

config = GRPOConfig(
    num_generations=8,
    beta=0.04,
)
# Override the loss computation to use distributional clipping:
# clip when TV(pi_new || pi_old) > delta, rather than when
# pi_new/pi_old exceeds [1-eps, 1+eps]
```

DPPO is Research-Stage

DPPO is a recent research contribution and is not yet integrated into mainstream RL libraries. It is most useful when you observe that standard ratio clipping is failing (e.g., on tasks with highly skewed token probability distributions).

7.5.10 ScaleRL and CISPO

Scaling Laws for RL

The ScaleRL paper [242] conducts a systematic study of what makes RL training for LLMs scale effectively. The key finding is that two modifications – batch-level reward scaling and DAPO-style token-level loss – together unlock strong performance at scale, while neither alone is sufficient. CISPO (Clipped IS Policy Optimization) is the resulting algorithm.

Batch-Level Reward Scaling

Standard GRPO normalises rewards within a group of G completions for a single prompt. CISPO normalises rewards across the *entire batch*:

$$\hat{A}_i = \frac{r_i - \mu_{\text{batch}}}{\sigma_{\text{batch}} + \epsilon},$$

where μ_{batch} and σ_{batch} are computed over all rewards in the current training batch. This provides a more stable baseline and prevents any single prompt from dominating the gradient.

CISPO Loss

CISPO combines batch-level scaling with DAPO's token-level loss aggregation and asymmetric clipping:

$$\mathcal{L}_{\text{CISPO}} = -\frac{1}{\sum_{i,t} m_{i,t}} \sum_{i=1}^{G} \sum_{t=1}^{|o_i|} m_{i,t} \cdot \min\!\left(\rho_{i,t}\hat{A}_i,\ \text{clip}_{\text{DAPO}}(\rho_{i,t}, \hat{A}_i)\,\hat{A}_i\right),$$

where $m_{i,t}$ is the overlong-filtering mask.

CISPO in TRL

```python
from trl import GRPOConfig, GRPOTrainer

config = GRPOConfig(
    loss_type="cispo",
    scale_rewards="batch",          # batch-level reward normalisation
    mask_truncated_completions=True,
    epsilon=0.2,
    epsilon_high=5.0,               # epsilon_max for CISPO (ScaleRL paper)
    num_generations=8,
)

trainer = GRPOTrainer(
    model=model,
    reward_funcs=[reward_fn],
    args=config,
    train_dataset=dataset,
)
```

ScaleRL Key Findings

1. Batch-level reward scaling alone: modest improvement.
2. Token-level loss alone: modest improvement.
3. Both together: **synergistic** – significantly better than either alone.
4. Larger batch sizes benefit more from batch-level scaling.
5. CISPO is the recommended default for large-scale RL training.

7.5.11 GDPO – Group Reward-Decoupled Policy Optimization

The Multi-Reward Collapse Problem

In multi-objective RL (e.g., optimising for both correctness and format), standard GRPO normalises the *combined* reward. If one reward has much higher variance than another, it dominates the normalised advantage, effectively ignoring the other reward. This is *advantage collapse*: the low-variance reward contributes near-zero gradient. GDPO [437] normalises each reward *independently* before aggregating.

The core mechanism normalises each reward *independently* before aggregating:

$$\hat{A}_n^{(i)} = \frac{r_n^{(i)} - \mu_n}{\sigma_n + \epsilon}, \qquad \hat{A}^{(i)} = \sum_{n=1}^{N} w_n \hat{A}_n^{(i)},$$

where $r_n^{(i)}$ is the n-th reward for completion i, μ_n and σ_n are the mean and standard deviation of reward n within the group, and w_n are user-specified weights.

GDPO vs Standard Multi-Reward GRPO

- **Standard**: $\hat{A}^{(i)} = \frac{\sum_n w_n r_n^{(i)} - \mu_{\text{combined}}}{\sigma_{\text{combined}}}$. High-variance rewards dominate.
- **GDPO**: normalise each reward separately, then combine. Each reward contributes proportionally to its weight w_n.
- GDPO is essential when rewards have very different scales or variances.

GDPO in TRL

```
from trl import GRPOConfig, GRPOTrainer

config = GRPOConfig(
    multi_objective_aggregation="normalize_then_sum",
    reward_weights=[1.0, 0.5],   # weights for [correctness, format]
    num_generations=8,
)

def correctness_reward(completions, **kwargs):
    return [1.0 if is_correct(c) else 0.0 for c in completions]

def format_reward(completions, **kwargs):
    return [0.1 if has_good_format(c) else 0.0 for c in completions]

trainer = GRPOTrainer(
    model=model,
    reward_funcs=[correctness_reward, format_reward],
    args=config,
    train_dataset=dataset,
)
```

7.5.12 GOPO – Group Ordinal Policy Optimization

GOPO [54] starts from a simple observation: reward models are trained with pairwise comparisons ("is A better than B?"), so only the **rank order** of their outputs is trustworthy—the raw numeric scores carry no inherent meaning. Yet GRPO feeds those raw magnitudes directly into the advantage calculation. For tasks with non-verifiable rewards—summarization, open-ended chat, instruction following—this mismatch introduces noise, because a gap of 0.6 reward points might reflect genuine quality in one region of the output space and mean nothing in another.

Key Insight: Discard reward magnitudes entirely. Use only the **ordinal ranking** of rewards within a group.

Algorithm: Given a group of N responses $\{o_1, \ldots, o_N\}$ with rewards $\{r_1, \ldots, r_N\}$: